axiateam
RDT company

Módulo 7

Contenedores con ECS/EKS

Curso: AWS Academy Cloud Foundations

5 horas

Objetivos de Aprendizaje

- **Explicar** los conceptos fundamentales de contenedores y Docker
- **Desplegar** aplicaciones en contenedores usando Amazon ECS
- **Gestionar** imágenes de contenedores con Amazon ECR
- **Implementar** contenedores serverless con AWS Fargate
- **Desplegar** y gestionar clusters de Kubernetes con Amazon EKS
- **Configurar** servicios de monitoreo para contenedores

1. Introducción a Contenedores

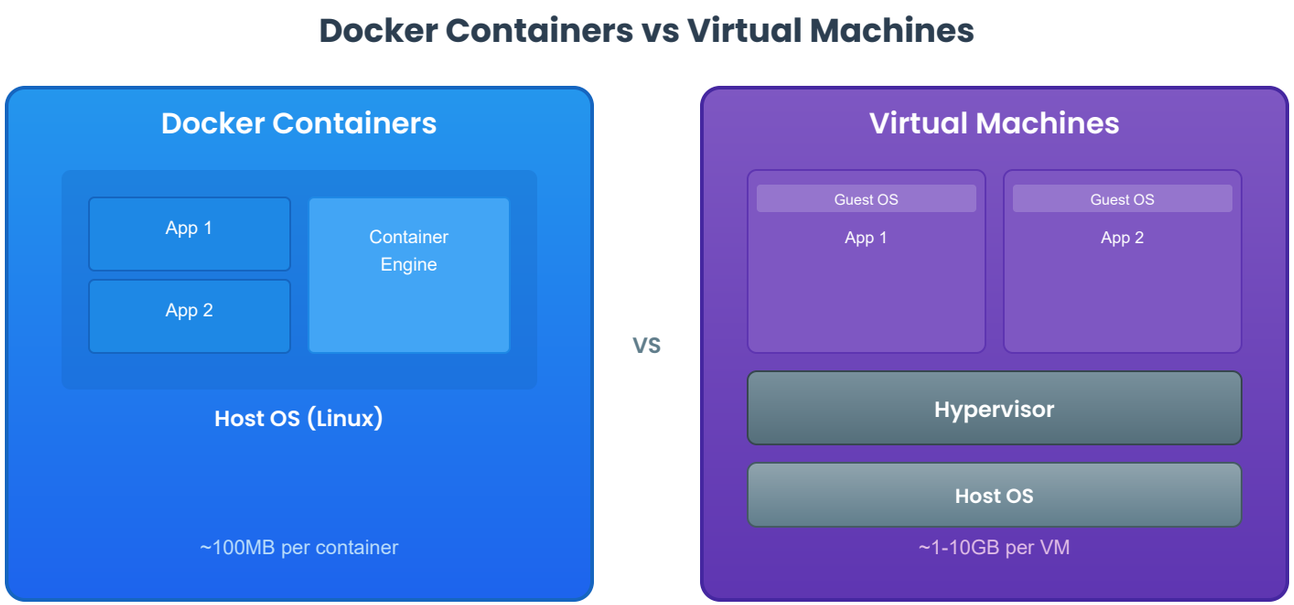
Docker, imágenes y contenedores

¿Qué es un Contenedor?

- Tecnología de virtualización a nivel de SO
- Aísla aplicaciones y sus dependencias
- Ligero comparado con VMs
- Consistente entre entornos

Key Point: Los contenedores comparten el kernel del host, lo que los hace mucho más eficientes que las VMs.

Docker vs Máquinas Virtuales



Containers: Mas ligeros, mayor densidad | VMs: Mayor aislamiento, mas overhead

Beneficios de Contenedores

Beneficio	Descripción
Consistencia	Mismo ambiente dev, test, prod
Eficiencia	Mayor densidad que VMs
Aislamiento	Apps independientes
Portabilidad	Corre en cualquier lugar con Docker
Escalabilidad	Rápido launch/stop

Componentes de Docker

Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["python", "app.py"]
```

Componentes de Docker

Docker Image vs Container

- **Docker Image:** Template de solo lectura con capas de filesystem
- **Docker Container:** Instancia en ejecución de una imagen
- El container tiene una capa rw sobre la imagen
- Los contenedores son efímeros por defecto

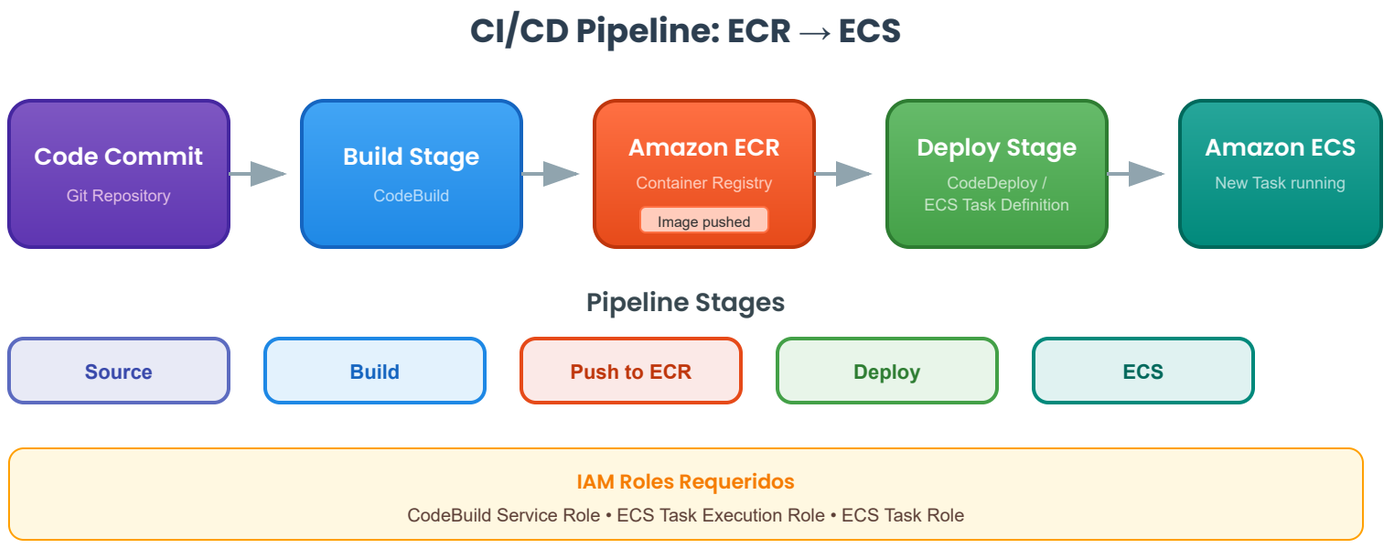
2. Amazon ECR

Elastic Container Registry - Registro de contenedores gestionado

¿Qué es Amazon ECR?

- Registro de contenedores **fully managed**
- Integrado con ECS y EKS
- Seguridad con IAM
- Escaneo de vulnerabilidades
- Replicación entre regiones

Flujo Básico con ECR



Crear Repositorio ECR

```
aws ecr create-repository \
    --repository-name my-webapp \
    --image-scanning-configuration scanOnPush=true \
    --encryption-configuration encryptionType=AES256
```

Taggear y Push Imagen

```
# Login
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin \
  <account>.dkr.ecr.us-east-1.amazonaws.com

# Tag
docker tag my-webapp:latest \
  <account>.dkr.ecr.us-east-1.amazonaws.com/my-webapp:latest

# Push
docker push <account>.dkr.ecr.us-east-1.amazonaws.com/my-webapp:latest
```

Lifecycle Policies

- Automatiza limpieza de imágenes no usadas
- Ahorra almacenamiento
- Previene acumulación de imágenes obsoletas

```
{
  "rules": [
    {
      "rulePriority": 1,
      "description": "Keep last 10 images",
      "selection": {
        "tagStatus": "tagged",
        "tagPrefixList": ["v"],
        "countType": "imageCountMoreThan",
        "countNumber": 10
      },
      "action": {
        "type": "expire"
      }
    }
  ]
}
```

Image Scanning y Security

Tipo	Descripción
Basic	Escaneo básico, incluido gratis
Enhanced	Escaneo continuo, requiere AWS Inspector

Nota: Scan on push está disponible automáticamente.

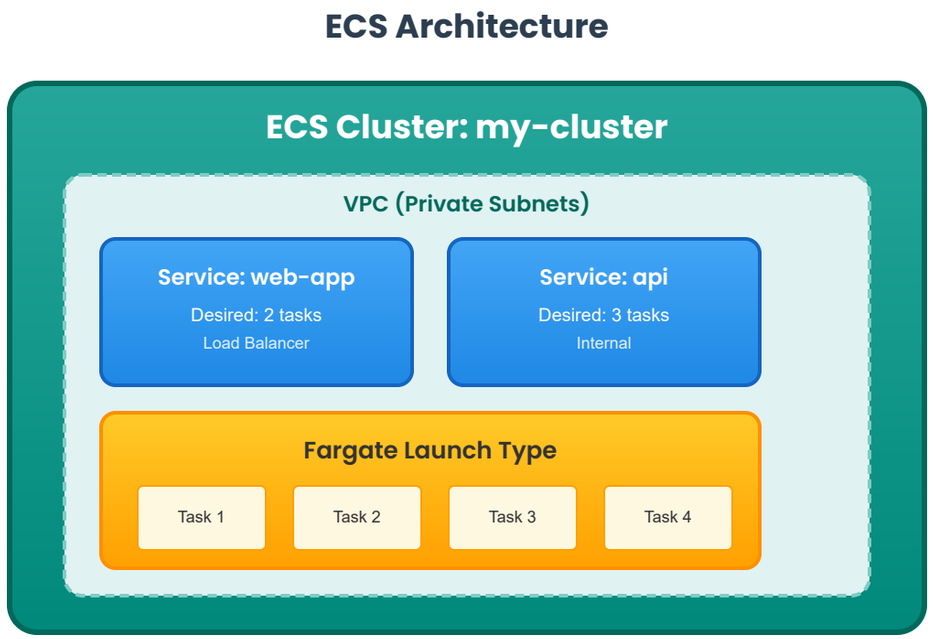
Cross-Region Replication

- Replicación automática a otras regiones
- Configurado a nivel de repositorio
- Útil para disaster recovery
- Reduce latencia para usuarios globales

3. Amazon ECS

Elastic Container Service - Orquestación de contenedores

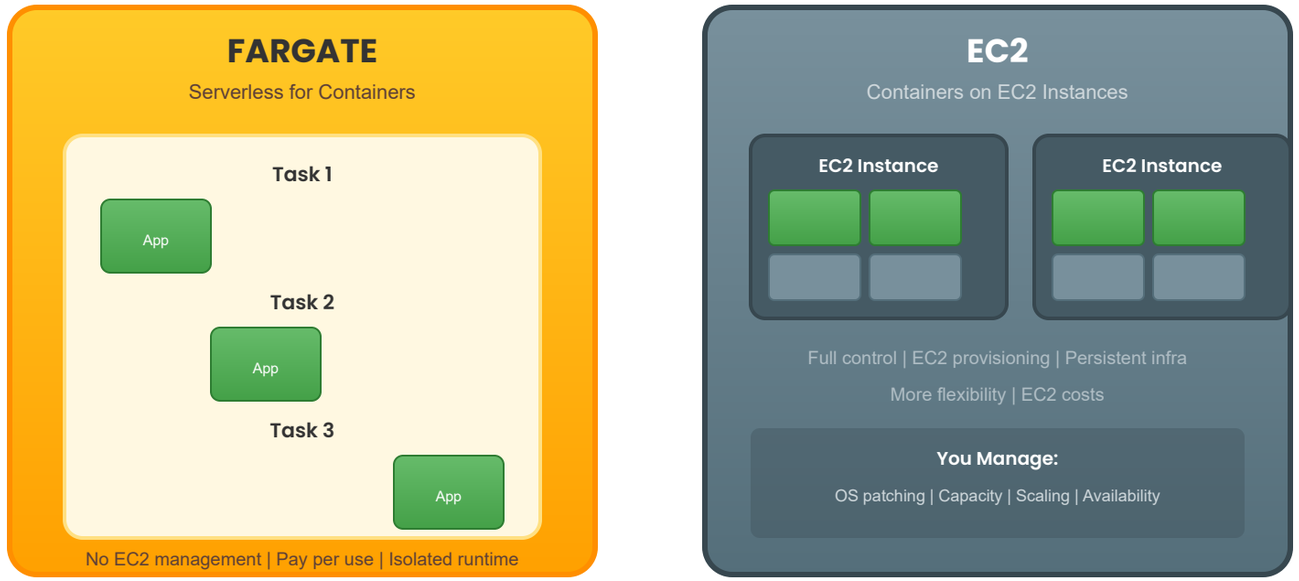
Arquitectura ECS



Launch Types: Fargate (serverless) | EC2 (self-managed) | External (on-premises)

Launch Types en ECS

Fargate vs EC2 Launch Type



Choose Fargate for less ops, EC2 for more control over infrastructure

Launch Types Comparativa

Type	Descripción	Casos de uso
Fargate	Serverless containers	Menos ops, cargas variables
EC2	Containers en EC2	Control de infra, workloads constants
External	On-premises	Hybrid cloud

Task Definitions

- Blueprint para tasks
- Define: image, CPU, memory, ports, environment
- Revision-based versioning

```
{
  "family": "webapp",
  "cpu": "256",
  "memory": "512",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "containerDefinitions": [...]
}
```

Task Placement Strategies

Estrategia	Descripción
AZ Balanced	Distribuye tasks entre AZs
Binpack	Empaqueta tasks en mínima cantidad de instancias
Random	Placement aleatorio

ECS Agent y Container Instances

- **ECS Agent:** Proceso que corre en cada EC2 instance
- Communica con ECS control plane

```
#!/bin/bash
echo "ECS_CLUSTER=my-cluster" >> /etc/ecs/ecs.config
yum install -y ecs-init
systemctl enable ecs
systemctl start ecs
```

Service Auto Scaling

Policy	Descripción
Target Tracking	Scale based on metric (CPU, Memory)
Step Scaling	Scale in steps based on CloudWatch
Scheduled Scaling	Scale at specific times

4. AWS Fargate

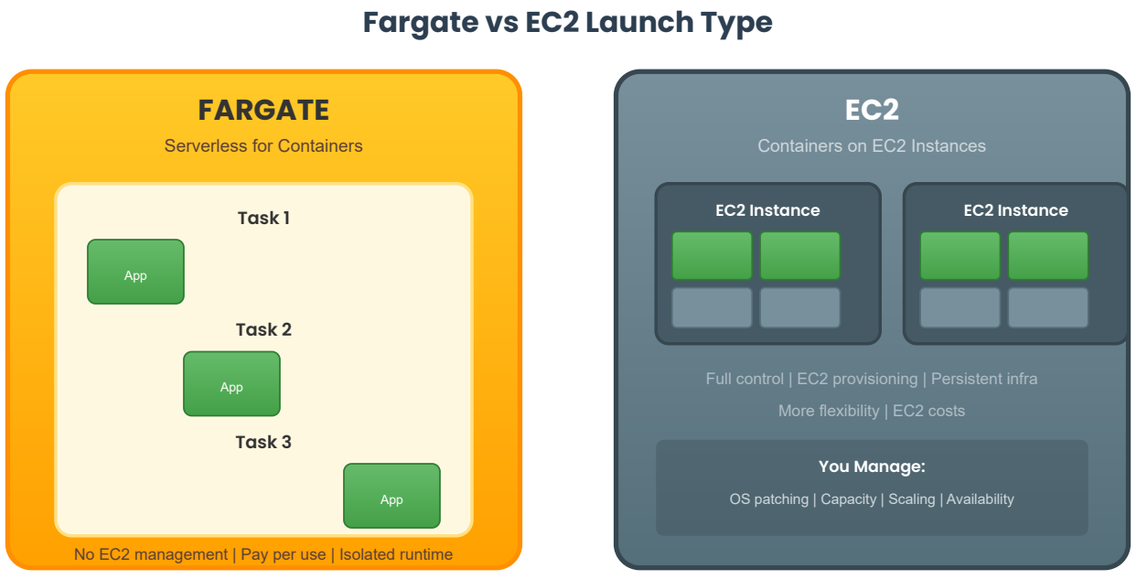
Serverless for containers

¿Qué es Fargate?

- Serverless for containers
- No EC2 instances to manage
- Pricing per vCPU/hour and GB/hour
- Automatic scaling

Importante: Fargate elimina la necesidad de gestionar servidores EC2 para containers.

Fargate vs EC2 Launch Type



Choose Fargate for less ops, EC2 for more control over infrastructure

Task Memory y CPU Allocation

vCPU	Memory
0.25	512MB, 1GB, 2GB
0.5	1GB, 2GB, 3GB
1	2GB - 8GB
2	4GB - 16GB
4	8GB - 30GB

Networking Modes

- **awsvpc** (Fargate only): Cada task tiene su propia ENI con IP única en VPC
- **bridge** (EC2 only): Docker bridge network con NAT
- **host** (EC2 only): Sin aislamiento de red

Best Practice: Usar awsvpc mode para máximo aislamiento y seguridad.

Lab 7.1

Desplegar Aplicación en Contenedores con ECS Fargate

Lab 7.1: Objetivo y Prerrequisitos

Objetivo: Desplegar una aplicación web en contenedores usando Amazon ECS con launch type Fargate

Duración: 60 minutos

Prerrequisitos:

- Cuenta AWS con permisos ECS, ECR, IAM, VPC
- Docker instalado localmente
- AWS CLI configurado

Lab 7.1: Pasos Principales

1. **Crear Repositorio ECR** - ECS > Repositories > Create
2. **Build y Push Imagen** - docker build, docker push
3. **Crear Cluster ECS (Fargate)** - VPC, subnets privadas
4. **Crear Task Definition** - Definir container, CPU, memory
5. **Crear Service** - Con ALB para balanceo
6. **Verificar Deployment** - Acceder via ALB DNS

Lab 7.1: Crear Cluster

```
# En ECS Console:
# Clusters > Create cluster

# Configuración:
# - Cluster name: my-cluster
# - VPC: Create new VPC
# - Subnets: 2 private subnets
# - Infrastructure: AWS Fargate
```

Lab 7.1: Task Definition

```
{
  "family": "webapp",
  "cpu": "256",
  "memory": "512",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "containerDefinitions": [
    {
      "name": "webapp",
      "image": "123456789.dkr.ecr.us-east-1.amazonaws.com/webapp:latest",
      "portMappings": [{
        "containerPort": 8000,
        "protocol": "tcp"
      }],
      "essential": true
    }
  ]
}
```

Lab 7.1: Verificación

```
# Obtener DNS del ALB
aws elbv2 describe-load-balancers \
  --names my-webapp-alb \
  --query 'LoadBalancers[0].DNSName' \
  --output text

# Acceder via navegador
# http://<alb-dns>:8000

# Ver tareas en ECS
aws ecs describe-services \
  --cluster my-cluster \
  --services my-webapp-service
```

Lab 7.2

Pipeline CI/CD para Imágenes con ECR y ECS

Lab 7.2: Objetivo y Prerrequisitos

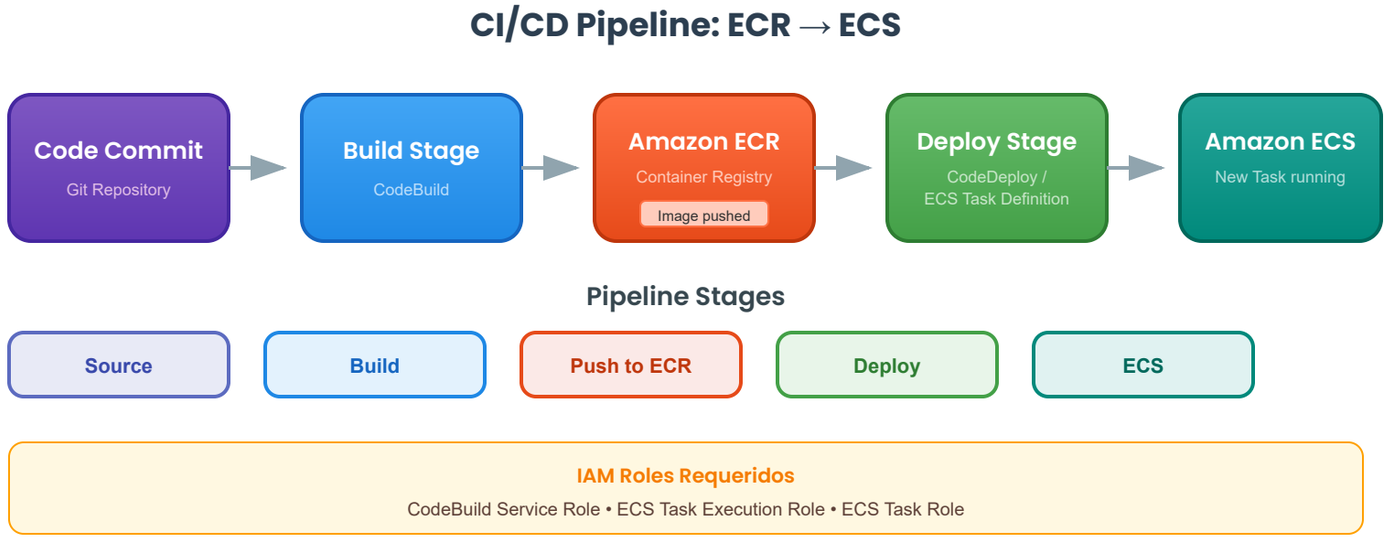
Objetivo: Crear un pipeline de CI/CD para construir y desplegar imágenes de contenedor automáticamente

Duración: 45 minutos

Prerrequisitos:

- Repositorio ECR del Lab 7.1
- Bucket S3 para artifacts
- CodePipeline y CodeBuild roles

Lab 7.2: Pipeline CI/CD



Lab 7.2: buildspec.yml

```
version: 0.2

phases:
  pre_build:
    commands:
      - echo Logging in to Amazon ECR...
      - aws ecr get-login-password --region $AWS_REGION | docker login --username AWS --password-stdin $AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazo
  build:
    commands:
      - echo Build started on `date`
      - docker build -t $IMAGE_REPO_NAME:$IMAGE_TAG .
      - docker tag $IMAGE_REPO_NAME:$IMAGE_TAG $AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/$IMAGE_REPO_NAME:$IMAGE_TAG
  post_build:
    commands:
      - echo Pushing the Docker image...
      - docker push $AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/$IMAGE_REPO_NAME:$IMAGE_TAG
```

Lab 7.2: CodePipeline Stages

1. **Source:** CodeCommit - Detecta cambios en código
2. **Build:** CodeBuild - Construye imagen Docker
3. **Deploy:** ECS - Actualiza servicio con nueva imagen

Lab 7.2: Trigger Pipeline

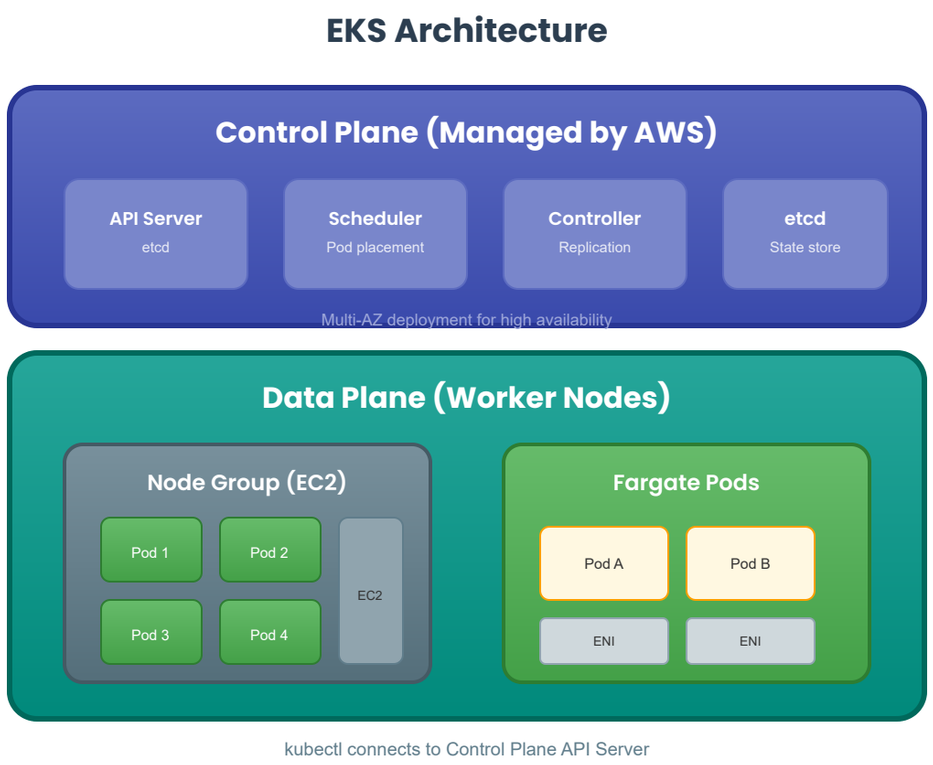
```
# Hacer commit a repositorio CodeCommit
git add .
git commit -m "Update webapp"
git push origin main

# Monitorear pipeline en CodePipeline console
```

5. Amazon EKS

Elastic Kubernetes Service - Kubernetes gestionado por AWS

Arquitectura EKS



Control Plane vs Data Plane

- **Control Plane:** Gestionado por AWS, Multi-AZ, etcd replicas
- **Data Plane:** Worker Nodes (EC2 o Fargate), donde corren los Pods

Node Groups

Type	Descripción
Managed Node Groups	AWS manages lifecycle, updates, patching
Self-managed Nodes	User gestiona EC2, más control
Fargate	Serverless pods, sin gestión EC2

kubectl y kubeconfig

```
# Actualizar kubeconfig
aws eks update-kubeconfig --name my-cluster --region us-east-1

# Verificar conexión
kubectl get nodes

# Obtener pods
kubectl get pods -A
```

Comandos kubectl Comunes

```
# Gestionar deployments
kubectl get deployments
kubectl describe deployment nginx
kubectl scale deployment nginx --replicas=3

# Gestionar pods
kubectl get pods
kubectl logs -f pod-name
kubectl exec -it pod-name -- /bin/bash

# Gestionar servicios
kubectl get services
kubectl expose deployment nginx --port=80 --type=LoadBalancer
```

Kubernetes Workloads - Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
    spec:
      containers:
      - name: webapp
        image: nginx:latest
        ports:
        - containerPort: 80
```

Kubernetes Workloads - Service

```yaml
apiVersion: v1
kind: Service
metadata:
  name: webapp-service
spec:
  type: LoadBalancer
  selector:
    app: webapp
  ports:
  - port: 80
    targetPort: 80
```

Fargate en EKS

```
aws eks create-fargate-profile \
    --cluster-name my-cluster \
    --fargate-profile-name my-fargate-profile \
    --pod-execution-role-arn arn:aws:iam::123456789:role/eks-fargate-pod-execution-role \
    --selectors namespace=default,labels={app=webapp}
```

Lab 7.3

Configurar Cluster EKS Básico con kubectl

Lab 7.3: Objetivo y Prerrequisitos

Objetivo: Crear un cluster Amazon EKS y desplegar una aplicación básica usando kubectl

Duración: 45 minutos

Prerrequisitos:

- eksctl instalado
- kubectl instalado
- IAM role con permisos para EKS
- VPC con subnets privadas

Lab 7.3: Crear Cluster con eksctl

```yaml
# cluster.yaml
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig

metadata:
  name: my-cluster
  region: us-east-1
  version: "1.29"

vpc:
  cidr: 10.0.0.0/16
  nat:
    gateway: HighlyAvailable

managedNodeGroups:
  - name: workers
    instanceType: t3.medium
    desiredCapacity: 2
    minSize: 1
```

```bash
# Crear cluster
eksctl create cluster -f cluster.yaml
```

Lab 7.3: Configurar kubectl

```
# Actualizar kubeconfig
aws eks update-kubeconfig --name my-cluster

# Verificar conexión
kubectl get nodes
```

Lab 7.3: Deployment y Service

```yaml
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
    spec:
      containers:
      - name: webapp
        image: nginx:latest
        ports:
```

Lab 7.3: Aplicar y Verificar

```
# Apply deployment
kubectl apply -f deployment.yaml

# Check status
kubectl get deployments
kubectl get pods
kubectl get services

# Get external URL
kubectl get services webapp-service

# Test access
curl http://<EXTERNAL-IP>
```

Lab 7.3: Cleanup

```
# Delete resources
kubectl delete -f deployment.yaml

# Delete cluster
eksctl delete cluster -f cluster.yaml
```

Resumen

Puntos Clave del Módulo 7

- **Contenedores:** Virtualización a nivel de SO, más eficientes que VMs
- **Docker:** Platform para crear, desplegar y ejecutar contenedores
- **Amazon ECR:** Registro de contenedores gestionado, integrado con IAM
- **Amazon ECS:** Orquestación de contenedores con Fargate o EC2
- **AWS Fargate:** Serverless containers sin gestión de EC2
- **Amazon EKS:** Kubernetes gestionado por AWS
- **CI/CD:** CodePipeline y CodeBuild para automatizar despliegues

Referencias

Recursos Adicionales

- AWS ECS Documentation: https://docs.aws.amazon.com/ecs/
- AWS ECR Documentation: https://docs.aws.amazon.com/ecr/
- AWS EKS Documentation: https://docs.aws.amazon.com/eks/
- Docker Documentation: https://docs.docker.com/
- Kubernetes Documentation: https://kubernetes.io/docs/
- eksctl Documentation: https://eksctl.io/

Gracias!

Módulo 7 Completado

Siguiente: Módulo 8 - Networking Avanzado